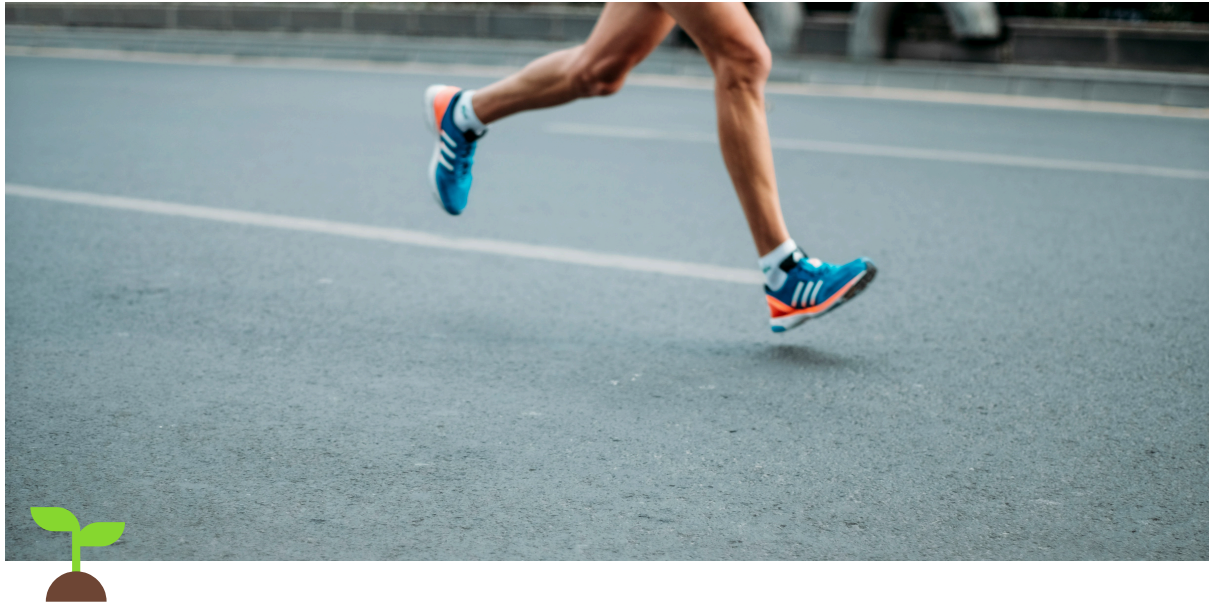




Code Structure

▼ Notazione

- **Attività:** Insieme di informazioni relative ad un'attività di corsa, principalmente distanza e percorso
- **Percorso:** Tragitto percorso dall'atleta, rappresentato come una linea spezzata i cui vertici sono tutte le coordinate rilevate durante l'attività.
- **Endpoints:** Punti di inizio o fine del percorso di un'attività
- **Atleta:** Utente bersaglio, del quale abbiamo a disposizione un set più o meno vasto di Attività.
- **EPZ**(Endpoint Privacy Zone): Area di privacy in cui vengono nascosti gli endpoint delle attività

▼ Threshold

- **distanceThreshold:** Distanza massima entro i quali due possibili circonferenze EPZ possono essere considerate uguali.
 - **intersectionThreshold:** Scarto tra la distanza centroEPZ-endpoint che possiamo accettare oltre al quale l'endpoint viene considerato interno alla circonferenzaEPZ e quindi quest'ultima eliminata.
 - **confidenceThreshold:** Numero minimo di ripetizioni che un possibleEPZ deve avere per non essere scartato.
 - **tau_converged:** Determina di quanto i centroidi posso spostarsi al massimo per considerare compiuto il ciclo di clustering per trovare gli EPZ nel secondo attacco
 - **tau_disjoint:** Determina la massima distanza che i punti all'interno di un cluster può avere con il suo centro.
-

▼ Data Flow

▼ Raccoglimento dei dati

I dati vengono raccolti tramite la procedura contenuta in *DataCollector/main.py*.

Si inizia con la creazione di un'istanza di *./Utility/ApplicationInfo()*, che legge da un file *.env* le variabili di sistema relativi e all'applicazione istanziata (Strava)

Si crea poi un istanza di *./DataController/AuthController()*, necessario a creare la sessione di autenticazione con l'applicazione, tramite le informazioni precedentemente caricate nell'*ApplicationInfo*

Una volta avviata l'istanza di connessione con l'applicazione, si crea un istanza di

`./DataCollector/ActivityRetrive()` per avviare la procedura di ottenimento di tutte le attività.

La procedura (`.fetchActivity()`) inizia ottenendo la lista degli ID di tutte le attività dell'utente, per poi scaricarne una ad una e salvarle nella cartella adatta (`"Data/Strava/UserID"`)

▼ Data Pre-Processing

Dato che il dataset era composto di circa 200 attività, le quali appartenenti anche a luoghi ben distanti da loro, è stato fatto un primo clustering. La logica con la quale sono state raggruppate è tramite semplice distanza tra endpoint di attività. Tutte quelle con uno dei due endpoint distanti meno di un certo valore da un attività 'modello' per il cluster, vengono associate al cluster. Quelle che non sono associate ad un cluster, diventano l'attività 'modello' per un nuovo cluster.

I cluster, rappresentati dai Model **ActivityCluster**, sono stati ottenuti dal metodo `clusterAllActivities()` presente in `./main.py`. Questo metodo prende in input il percorso dell'utente del quale si vogliono clusterizzare, e di quale le attività si troveranno nella sottocartella `./activities/`. I vari cluster sono poi stati salvati all'interno del file `ActivityClusterList.json` presente nella cartella del relativo utente. Ad ogni cluster è stato assegnato un 'id' incrementale il quale viene usato per l'istanziatura di un `ActivityCluster`, insieme al suo percorso, dal metodo `initializeActivityClusterFromJson()`.

▼ Difesa

Il primo livello di difesa, prende ogni attività ed elimina ricorsivamente gli endpoint che sono ad una distanza dal punto scelto come centro dell'EPZ inferiore al suo raggio.

Un livello successivo di difesa, prevede che la distanza venga calcolata rispetto ad un punto spostato dal centro dell'EPZ.

Un supplemento a questa difesa prevede l'aggiunta di un punto esattamente alla distanza pari al raggio.

(Identificheremo come scelta base quella di aggiungere questo punto, e versione con "Fuzz" quella in cui questo punto viene 'eliminato')

La routine è contenuta nel file `./main.py`, costituita dalla funzione `fullDisguiseOfActivityCluster()`. La routine prima inizializza un `activityCluster` (che contiene tutte le attività relative), istanzia due oggetti **DataRepresentation** (sferico e geocentrico), e in fine cicla per ogni attività contenute nel cluster chiamando il metodo (relativo alla classe `Activity`) `.completeDisguiseActivityInCluster()`.

Questo metodo esegue tutti i tipi di difesa possibili, quindi: Primo livello con e senza Fuzz e secondo livello con e senza Fuzz. Queste difese vengono fatte sia con la rappresentazione dei dati sferica che geocentrica. Ognuna di queste difese darà come risultato una polilinea, che viene codificata e salvata come testo nel file relativo dell'attività.

▼ Attacco di primo livello

La routine dell'attacco di primo livello è implementato nel file `./main.py`, come funzione `attack()`. Questa funzione prende in ingresso il percorso delle attività da attaccare, la classe `DataRepresentation` nella quale si vuole eseguire l'attacco, e il tipo di attacco, come stringa. Questa routine sfrutta la classe **EPZSearch** del modulo **Attack1**, chiamando sequenzialmente le funzioni necessarie ad eseguire l'attacco.

Dopo l'istanziatura di **EPZSearch**, la sequenza è la seguente:

- `initializeAttack()`: Vengono ciclata tutte le coppie di endpoint, e per ognuna di queste viene trovato il possibile centro dell'EPZ (Quindi un punto a distanza data da entrambi gli endpoint) tramite `.getPossibleEPZfromPointPair()` della classe

DataRepresentation. I possibili centri EPZ vengono istanziati come classi **possibleEPZ** (sottoclasse di **EPZ**), e aggiunti al dizionario .possibleEPZs

- deleteEPZintersectingActivity(): Vengono eliminati tutti i *possibleEPZ* (immaginabili come una circonferenza, con un centro e un raggio), che hanno al loro interno spaziale almeno un endpoint.
- groupCloseEPZs(): Vengono raggruppati insieme i *possibleEPZ* con raggio uguale e centro ad una distanza inferiore di un certo distanceThreshold.
- deleteInformationlessEPZ(): Si scartano tutti i *possibleEPZ* con una ripetizione inferiore ad un certo confidenceThreshold.
- convertInLatLon(): Si convertono i risultanti EPZ in coordinate lat-longitudinali.

▼ Attacco di secondo livello

La routine dell'attacco di secondo livello è implementato nel file *./main.py*, come funzione attack2(). Questa funzione prende in ingresso il percorso della lista di cluster e l'id del cluster. La routine sfrutta la classe **EPZSearch** del modulo **Attack2**. Questa viene inizializzata con il cluster di attività, processo nel quale si sfrutta il metodo initActivityEndpointList() della rappresentazioni dati che si vuole utilizzare. Questo metodo prepara tutti gli endpoint necessari per eseguire l'attacco: per ogni attività, partendo dal primo nodo, controlla quali sono (consecutivamente) all'interno dell'EPZ, salvandosi in un attributo dell'endpoint finale quale è la distanza cancellata cumulativamente.

L'inizializzazione degli endpoint così fatta è come se mettesse insieme la difesa di secondo tipo e l'inizio dell'attacco.

L'attacco procede poi chiamando il metodo epz_identification(), al quale vengono passati due threshold usati per identificare i centri e i raggi delle

zone EPZ dentro il quale bisognerà trovare il punto d'interesse. Questo metodo sfrutta una variante del metodo *k-means*.

Infine, utilizza il metodo *retriveSensitiveLocation()* per determinare il punto di interesse all'interno dell'EPZ. Questo metodo inizia scaricando il grafo della rete stradale. Poi, cerca, per ogni endpoint il nodo stradale più vicino. Segue inizializzando un DataFrame delle distanze, e calcolando per ogni nodo-endpoint la distanza necessaria per arrivare ad ogni nodo del grafo.

A questo punto, tramite un algoritmo DBSCAN, si cercano gli Entry Gates, ovvero cluster di nodi vicini che costituiscono i punti di accesso alla zona di privacy, per poi eliminare tutti i nodi outlier.

A questo punto si procede con il trovare il punto di interesse: Per ogni nodo-endpoint, si prende la distanza associata (quanto percorso è stato cancellato dalla difesa per ottenere quel nodo come primo) e la si sottrae a tutte le distanze che quel nodo ha da ogni nodo del grafo. Si sommano poi tutte le distanze per nodi uguali, e quella con il valore minore sarà il nodo di nostro interesse.

▼ Raccolta risultati e Statistiche

Gli attacchi sono poi stati testati sul nostro dataset, i risultati collezionati ed analizzati; queste procedure sono contenute nel file *./stats.py*. Data la limitatezza del dataset, è stato eseguito un processo di *bootstrapping*, ovvero una tecnica che prevede di prendere dei sottocampioni casuali, con ripetizioni, per permettere di irrobustire la stima del test. Questo è fatto nella prima parte, ovvero quella di collezione dei dati, eseguiti tramite le funzioni *retriveStats()* e *retriveStatsSecondAttack()* rispettivamente per il primo e secondo attacco. Queste funzioni simulano la difesa di un batch casuale di attività, scegliendo un punto casuale come centro della difesa, per poi attaccare tramite le funzioni viste fino ad ora. I risultati, salvati in

opportuni file .csv nella cartella *./Data/Stats/*, sono poi analizzati tramite la funzione *calculateStats()* e *calculateStats2nd()*.

▼ **Struttura**

▼ *./DataCollector/*

▼ *Strava/*

▼ **StravaSession.py**

Sessione specializzata per l'applicazione di Strava, con funzione *.get()* tale da controllare i limiti di richieste effettuabili

▼ **ActivityRetriver.py**

Inizializzato tramite una sessione OAuth2.0 attiva, ne si utilizza la funzione *.fetchActivity()* per scaricare tutte le attività di un determinato utente.

▼ **AuthController.py**

Controller della sessione di autenticazione tra utente terzo e applicazione, necessaria per ottenere le attività. Si occupa di avviare la sessione e comunicare con l'utente durante il processo di autenticazione.

▼ **main.py**

Contiene la procedura completa per recuperare tutte le attività

▼ **SessionFactory.py**

Factory della sessione OAuth2.

▼ *./Models/*

▼ **ActivityCluster.py**

Modello per contenere attività Clusterizzate. Contiene diversi attributi relativi al cluster, come il centro o

il centro shiftato (usato nella difesa), e la lista delle attività.

Il modello si avvale di un Object-Document Mapping (ODM) per gestire la persistenza dei dati nei file `ActivityClusterList.json`; in particolare sono presenti i metodi `initializeActivityClusterFromJson()` per la lettura, `addClusterToActivityClusterList()` per aggiungere un cluster alla lista e `updateActivityListToActivityClusterJson()` per aggiornare la lista delle attività del relativo cluster.

Presenta poi i seguenti metodi:

- `initializeActivityList()`: Per istanziare tutti gli oggetti attività ed aggiungerli alla lista presente nell'attributo `.activityList`.
- `addCircleToPlot()`: Passando un plot di tipo `matplotlib.pyplot`, aggiunge la circonferenza relativa all'`ActivityCluster`.
- `getAllClusters()`: Dato il percorso che contiene le attività, ed un raggio, raggruppa in cluster tutte le attività che iniziano ad una distanza minore del raggio data. Ha un parametro facoltativo, `clusterThreshold`, che permette di impostare il minimo di attività che un cluster deve avere per essere considerato; di default è 3. Ha, infine, un parametro booleano, `checkEnd`, che permette di decidere se controllare anche la fine dei percorsi, per tutti quelli che non è stato trovato un cluster abbastanza popoloso (`clusterThreshold`).
- `updateActivityList()`: Funzione utilizzata per aggiornare la lista di attività del cluster, nel caso in cui vengano aggiunte nuove attività alla lista.
- `updateActivityClusterCenterInJson()`: Modifica il centro del cluster nel file json che contiene la lista dei cluster.

▼ PossibleEPZ.py

Modello per contenere attività Clusterizzate. Contiene diversi attributi relativi all'EPZ relativo, come il centro o il centro shiftato, e la lista delle attività.

Presenta il metodo *initializeEPZfromJson()* per istanziare oggetti tramite il percorso del file epzlist.json. Contiene poi metodi che permettono di ottenere i cluster o modificare la lista di EPZ.

Contiene infine una sottoclasse, **possibleEPZ**, usata durante gli attacchi per contenere le informazioni dei possibili centri di privacy trovati. Essa contiene metodi

▼ Activity.py

Modello che rappresenta un'Attività. Contiene le informazioni più importanti delle attività, tra cui le polilinee che rappresentano il percorso. Le polilinee sono salvate in un dizionario nell'attributo *.maps*, e sono salvate come stringhe codificate tramite l'algoritmo di Google.

Il modello si avvale di un Object-Document Mapping (ODM) per gestire la persistenza dei dati nei file .json; in particolare sono presenti i metodi *initActivityFromPath()* per la lettura e *updateActivityJson()* per la scrittura.

Sono presenti poi metodi per applicare l'algoritmo di difesa all'attività:

- *disguiseActivityInCluster()*: Applica un solo tipo di difesa (primo o secondo livello, con o senza *fuzz*, e per una sola Rappresentazione dei Dati), salvandolo nell'attributo *maps*.

Prende in ingresso il cluster di attività, la *dataRepresentation* da usare, e 3 parametri booleani (*fuzz*, *cloacked*, *updateJson*): i primi due per impostare il tipo di attacco (con o senza *fuzz*, centro shiftato oppure no), e per impostare se

aggiornare o meno il file json. Di default sono tutti False.

- `completeDisguiseActivityInCluster()`: Applica tutte le difese, salvandole nell'attributo maps. Prende in ingresso il centro, il centro shiftato, il raggio, e due istanze diverse di `dataRepresentation`

Ci sono poi metodi per codificare e decodificare le polilinee (`encodePolyline()` e `decodePolyline()`) e metodi per plottare l'attività.

▼ **Point.py**

Classe Model dei punti di coordinate relative al **DataRepresentation**

▼ **User.py**

Classe Model dell'utente Strava, utilizzato per gestire la cartella

▼ ***./DataRepresentation/***

▼ **DataRepresentation.py**

La classe astratta `DataRepresentation` serve come interfaccia per costruire delle specializzazioni di rappresentazioni dei dati. Esse fungono da container per tutti i metodi specifici della tipologia di rappresentazione del dato usata.

Ci sono 3 specializzazioni:

- **`SphericalDataRepresentation()`**: Utilizza coordinate latlon e ha metodi che si basano sulla geometria geodetica della terra.
- **`GeocentricDataRepresentation()`**: Utilizza coordinate x,y ottenute come omeomorfia di quelle lat-longitudinali su un piano, e utilizzano metodi geometrici planari.
- **`UTMDataRepresentation()`**: Utilizzano il sistema *UTM*, un altro sistema di proiezione sul piano, con metodi geometrici planari.

Le prime due sono utilizzate per attuare l'attacco di primo livello, e contengono gli stessi metodi:

- *getPolylineList()*: Data il percorso di una cartella contenente molteplici attività, restituisce una lista di dizionari {"id": *int*, "polyline": [(*int,int*)]}, con polyline lista di punti nella corrispettiva rappresentazione.
- *getActivityEndpointList()*: Dato il percorso di una cartella contenente molteplici attività, restituisce una lista con tutti gli *Endpoint* delle suddette attività.
- *getPossibleEPZfromPointPair()*: Metodo utilizzato nell'attacco di primo livello che, dati due punti, trova i due centri delle circonferenze che si trovano a distanza data e passano esattamente per quei due punti.
- *distance()*: Metodo per trovare la distanza per il metodo di rappresentazione in uso
- *midPoint()*: Metodo per trovare il punto a metà tra due punti
- *getPointOnCircumference()*: Dato il centro di una circonferenza ed un punto esterno, restituisce il punto sulla circonferenza nella direzione centro-punto.
- *generateCloackedCenter()*: Dato un centro e un raggio, restituisce un punto casuale all'interno della circonferenza, a distanza compresa tra [0.4*raggio,0.9*raggio] dal centro

L'ultima rappresentazione dei dati è usata per l'attacco di secondo livello. Contiene la definizione della classe **UTMEndpoint()**, più alcuni metodi utili all'attacco:

- *utmDistance()*: Restituisce la distanza tra due utmPoint

- `getActivityEndpointList()`: Dato il percorso di una cartella contenente molteplici attività, restituisce una lista di Endpoint nella relativa rappresentazione
- `initActivityEndpointList()`: Dato un `activityCluster`, vengono inizializzate tutte gli endpoint delle attività nel cluster. In particolare, per ogni attività, vengono eliminati tutti i punti agli estremi dell'attività che risultano all'interno della circonferenza shiftata dell'EPZ, salvando cumulativamente la distanza cancellata.

▼ **DataRepresentationFactory.py**

Contiene le classi relative al design pattern *Factory Method* per istanziare le classi *DataRepresentation*.

▼ **./Attack1/**

▼ **EPZSearch.py**

Classe che contiene gli elementi necessari a eseguire l'attacco di primo livello. Per istanziare è necessario un oggetto *dataRepresentation*, la lista di endpoint su cui fare l'attacco, le informazioni dell'applicazione (per i vari threshold) e il tipo di attacco. Durante l'istaziamento viene creata una lista contenente tutte le possibili coppie di Endpoint, oltre che caricate tutte le variabili d'ambiente dell'attacco.

Contiene poi i seguenti metodi:

- `initWithPathList()`: Permette l'istanziazione tramite percorso della cartella che contiene le attività piuttosto che passare direttamente la lista degli endpoint delle attività.
- `getAllPairs()`: Funzione utilitaria che data una lista di elementi, crea una lista con tutte le coppie possibili dei dati elementi.
- `initializeAttack()`: Vengono ciclato tutte le coppie di endpoint, e per ognuna di queste viene trovato il possibile centro dell'EPZ (Quindi un punto a

distanza data da entrambi gli endpoint) tramite .getPossibleEPZfromPointPair() della classe `DataRepresentation`. I possibili centri EPZ vengono istanziati come classi **possibleEPZ** (sottoclasse di **EPZ**), e aggiunti al dizionario `.possibleEPZs`

- *deleteEPZintersectingActivity()*: Vengono eliminati tutti i *possibleEPZ* (immaginabili come una circonferenza, con un centro e un raggio), che hanno al loro interno spaziale almeno un endpoint.
- *groupCloseEPZs()*: Vengono raggruppati insieme i *possibleEPZ* con raggio uguale e centro ad una distanza inferiore di un certo `distanceThreshold`.
- *deleteInformationlessEPZ()*: Si scartano tutti i *possibleEPZ* con una ripetizione inferiore ad un certo `confidenceThreshold`.
- *convertInLatLon()*: Si convertono i risultanti EPZ in coordinate lat-longitudinali.
- *attack()*: Esegue in sequenza le funzioni che costituiscono nel complessivo l'attacco di primo livello. (`initializeAttack()`, `deleteEPZintersectingActivity()`, `groupCloseEPZs()` e `deleteInformationlessEPZ()`)

▼ *./Attack2/*

▼ **EPZSearch.py**

Classe che contiene gli elementi necessari a eseguire l'attacco di secondo livello. Si inizializza con un cluster di attività, e in questo processo viene inizializzata la lista di endpoint, tramite il metodo *initActivityEndpointList()* della classe

DataRepresentation.

Presenta poi metodi per impostare o leggere alcuni attributi, e per altri scopi:

- *euclidean_distance()*: Calcola la distanza tra due `UTMPoint`

- `fit_circle()`: Utilizzata da `epz_identification()` per trovare la circonferenza che aderisce ai punti dati
- `epz_identification()`: Raggruppa i punti in zone di circonferenza diverse, trovando per ognuna i centri delle circonferenze che li raggruppano. Metodo implementato come variante del *k-means*.
- `retriveSensitiveLocation()`: Metodo principale dell'attacco di secondo livello. Serve per determinare il punto di interesse all'interno dell'EPZ. Questo metodo inizia scaricando il grafo della rete stradale. Prosegue associando, ad ogni endpoint, il nodo stradale più vicino. Segue inizializzando un DataFrame delle distanze, e calcolando per ogni nodo-endpoint la distanza necessaria per arrivare ad ogni nodo del grafo. A questo punto, tramite una variazione dell'algoritmo DBSCAN, si cercano gli Entry Gates, ovvero cluster di nodi vicini che costituiscono i punti di accesso alla zona di privacy, per poi eliminare tutti i nodi outlier. A questo punto si procede con il trovare il punto di interesse: Per ogni nodo-endpoint, si prende la distanza associata (quanto percorso è stato cancellato dalla difesa per ottenere quel nodo come primo) e la si sottrae a tutte le distanze che quel nodo ha da ogni nodo del grafo. Si sommano poi tutte le distanze per nodi uguali, e quella con il valore minore sarà il nodo di nostro interesse.
- `enhance_graph()`: Prendendo in input il grafo stradale e la massima distanza che si vuole avere tra i due nodi, si aumenta la risoluzione di nodi per archi che hanno una lunghezza maggiore del valore dato.
- `plot_heatmap_over_osmnx()`: Metodo per plottare la heatmap sopra il grafo stradale.

- `plot_heatmap_clusters_over_osmnx()`: Metodo per plottare la heatmap e gli entry gates sopra il grafo stradale.
- `plotClusters()`: Metodo per plottare gli entry gates trovati.

▼ *./Data/*

Cartella che raggruppa tutti i dati persistenti usati nel progetto

▼ *./Stats/*

Cartella contenente i risultati delle statistiche. I file contenuti sono di tipo `.csv` e hanno come nome "Stats" seguito dalla data per gli attacchi di primo livello, "Stats2ndAttack" seguito dalla data per gli attacchi di secondo livello.

▼ *./Strava/*

Cartella contenente i dati dell'applicazione 'Strava', ha una gerarchia del tipo

- */userId/* : contiene tutte le informazioni dell'utente
 - */activities/* : cartella contenente tutte le attività dell'utente
 - **/ActivityClusterList.json** : file che contiene tutte le informazioni sui cluster dell'utente

▼ *./images/*

Cartella contenente tutte le immagini e i grafici plottati durante la realizzazione del progetto. La cartella è organizzata in sottocartelle chiamate con la data nella quale sono state create.

▼ *./PreProcessing/*

Contiene una serie di funzioni utilizzate per processare dei dataset secondari, ma non sono mai stati utili in quanto mancanti di informazioni.

▼ *./Utility/*

▼ **ApplicationInfo.py**

Contenitore delle informazioni riguardante l'applicazioni in esame. Vengono caricate tramite un file `.env` durante l'istanzamento della classe. Contiene informazioni quali url per la connessione, raggi d'azione possibili e threshold degli attacchi.

▼ **jsonHelper.py**

Contenitore di funzioni per leggere e scrivere file json.

Durante l'inizializzazione, si salva un parametro con una lista di tutti gli utenti per un determinato utente (tramite funzione **`getUserList()`**).

Impostando l'ID dell'utente (**`setCurrentUser()`**), o passandolo come parametro, si può ottenere la lista degli *ActivityID* (**`getActivityIdListByUserID()`**).

metodi Statici

- **`getActivityIdListByPath(path)`**: Restituisce una lista di *ActivityID* contenute nel percorso *path*
- **`getJsonValues(jsonPath,keys)`**: Restituisce, sottoforma di dizionario, i valori delle chiavi passate tramite *keys*, all'interno del file del percorso *jsonPath*.

▼ *./*

▼ **main.py**

Script principale del progetto che contiene tutte le routine utilizzate, raggruppate in funzioni.

▼ **DataCollection**

- *fetchDataFromApplication()*: Funzione che contiene la routine per scaricare tutte le attività di un utente, previa sua autorizzazione. Inizia istanziando il controller dell'autenticazione, avvia la sessione (punto in cui avviene l'autorizzazione da parte dell'utente in questione), istanzia l'oggetto *activityRetriver* per poi scaricare le attività.

▼ Difesa

- *signleDisguiseTest()*: Prende in ingresso il percorso di una lista di cluster, l'id del cluster da difendere e la *dataRepresentation* da usare, e viene applicato la difesa di primo livello con *Fuzz* alla prima attività del cluster. Prende un parametro booleano (*plot*) per poter plottare la polilinea con il relativo cerchio; di default è impostato su Falso.
- *disguiseActivityInCluster()*: Prende in ingresso il percorso di una lista di cluster, l'id del cluster da difendere e la *dataRepresentation* da usare, e viene applicato la difesa di primo livello con *Fuzz* alla prima attività del cluster. Prende un parametro booleano (*plot*) per poter plottare la polilinea con il relativo cerchio; di default è impostato su Falso.
- *fullDisguiseOfActivityCluster()*: Prende in ingresso il percorso di una lista di cluster e l'id del cluster da difendere, e applica la difesa completa (primo e secondo livello, con e senza *Fuzz*) ad ogni attività del cluster. Fa leva sul metodo *completeDisguiseActivityInCluster()* della classe *Activity*.

▼ Attacco di primo livello

- *attack()*: Esegue l'attacco di primo livello. Prende in ingresso il percorso dove sono presenti le attività da attaccare, la *datarepresentation*

da usare e il tipo di attacco da fare. Con questi inizializza la classe **EPZSearch** del modulo **Attack1**, e chiama tutti i metodi sequenzialmente per eseguire l'attacco. Stampa in console, per ogni passo intermedio, il tempo impiegato per il passo precedente e il numero di EPZ potenziali trovati fino a quel punto.

- testAttack(): Funzione che esegue l'attacco di primo livello, utilizzando la funzione attack(), e poi crea dei plot con i risultati ottenuti.

▼ Attacco di secondo livello

- secondAttack(): Prende in ingresso il percorso del cluster di attività, e l'id del cluster, ed esegue l'attacco di secondo livello. Dopo aver inizializzato il cluster, istanzia la classe **EPZSearch** del modulo **Attack2**, e avvia il metodo epz_identification() che inizia trovando le circonferenze, e infine retriveSensitiveLocation(), che trova il punto specifico.

▼ Altro

- compairDataRepresentation(): Prende in ingresso il percorso di una lista di cluster e l'id del cluster. La funzione estrae gli endpoint delle prime *firstN* attività del cluster, esegue la prima parte dell'attacco di primo livello con entrambe le *DataRepresentation*, e poi plotta tutte le attività, gli endpoint e le circonferenze trovate.
- collectImages(): Itera il primo attacco testando un insieme diversi di valori per i vari **threshold**, salvando l'immagine dei vari risultati. Utilizza la funzione testAttack().
- initializeApplication(): inizializza globalmente la variabile *appInfo*, che contiene le

informazioni e le costanti riguardante l'applicazione di fitness che si sta attaccando. Prende in ingresso la stringa con il nome dell'applicazione.

- *initializeUser()*: Istanza e inizializza un oggetto del modello User, passando l'id dell'utente come parametro.
- *addPolylineToPlot()*: Aggiunge, alla *matplotlib.pyplot* passato come parametro, la polilinea che rappresenta il percorso dell'attività.
- *plotUserActivity()*: Passando l'userid, plotta tutte le attività dell'utente.

▼ stats.py

Script utilizzato per eseguire più volte gli attacchi, salvarne i risultati per poi essere analizzati.

- *initializeApplication()*: inizializza globalmente la variabile *appInfo*, che contiene le informazioni e le costanti riguardante l'applicazione di fitness che si sta attaccando. Prende in ingresso la stringa con il nome dell'applicazione.
- *retriveStats()*: Routine che colleziona i risultati per il primo attacco. Cicla per tutti i cluster di attività, per ognuno di questi inizia leggendo tutte le attività e mettendole in una lista. Per ogni cluster, viene ciclato per tutti i raggi che l'EPZ può assumere per la data applicazione, e per ognuno dei raggi cicla un numero casuale di volte compreso tra 8 e 18. Il ciclo principale prevede un'estrazione casuale dalla lista di attività (*random_batch()*), l'attacco completo di primo livello e l'aggiunta dei risultati in un DataFrame. Alla fine di tutti i cicli, il DataFrame viene salvato in un file .csv

- retriveStatsWithClusters(): Routine che colleziona i risultati di una variante dell'attacco di primo livello. In particolare, invece di usare gli endpoint di tutte le attività, esegue un clustering di queste e utilizza un solo endpoint per cluster.
 - retriveStatsSecondAttack(): Routine che colleziona i risultati per il secondo attacco. La struttura è identica a quella del primo attacco (retriveStats()), con la differenza che viene utilizzato l'attacco di secondo livello.
 - random_batch(): Dando una lista, restituisce un batch casuale di quella lista.
 - disguiseActivityBatch(): Esegue la difesa di primo livello al batch di attività che viene passato, nel centro e con i raggi passati come parametri. Ha un ultimo parametro booleano (*fuzz*) per scegliere o meno se eseguire il Fuzz; falso di default.
 - str_to_array(): Trasforma un array scritto come stringa in un vero e proprio array.
 - find_cluster(): Dato una lista di endpoint, una distanza e un numero di *min_samples*, clusterizza gli endpoint restituendo un dataframe con il cluster a cui appartiene ogni endpoint.
 - calculateStats(): Calcola le statistiche dei file .csv ottenuti tramite le funzioni retriveStatsWithClusters()/retriveStats(). Calcola le percentuali di successo degli attacchi, divisi per raggio, per ogni file.
 - calculateStats2nd(): Calcola le statistiche dei file .csv ottenuti tramite le funzioni retriveStatsSecondAttack(). Calcola le percentuali di successo degli attacchi, divisi per raggio, per ogni file.
-